

MAJORITY ELEMENT

Boyer–Moore Voting Algorithm

20-slide interactive learning deck

Learn • Visualize • Code • Practice

START LEARNING →

CORE IDEA

Cancel different values.
The true majority survives.

Interactive Contents

Click any section to jump there.

01

02

03

04

05

06

07

08

Interview + revision

1. 🧠 What is Majority Element?

The definition is simple; the constraint makes the solution interesting.

DEFINITION

An element is a majority element if it appears more than $\lfloor n/2 \rfloor$ times.

EXAMPLE

[2, 2, 1, 1, 1, 2, 2]
2 appears 4 times $\rightarrow 4 > 7/2$

TARGET

Return the majority element. LeetCode 169 guarantees that one exists.

See the problem as a vote

Every occurrence is a vote for its value.



Majority = more than half the votes

We need a way to discard information safely.

2. ? Why do we need Boyer–Moore?

Compare the common approaches.

BRUTE FORCE

Check counts repeatedly.

Time $O(n^2)$
Space $O(1)$

HASH MAP

Store frequencies.

Time $O(n)$
Space $O(n)$

BOYER–MOORE

Cancel pairs.

Time $O(n)$
Space $O(1)$

Interview goal → keep the speed, remove the extra memory.

The intuition: cancellation

A majority has more votes than all other values combined.

PAIR 1

2 $\leftrightarrow$ 1

Different values cancel.

PAIR 2

2 $\leftrightarrow$ 1

Another pair cancels.

LEFT

2 remains.

WHY SAFE?

The majority cannot be fully cancelled.

3. Boyer–Moore in 3 rules

RULE 1

count == 0
→ candidate = x

RULE 2

x == candidate
→ count++

RULE 3

x != candidate
→ count--

Think of count as the candidate's unmatched lead.

Pseudocode

No sorting. No frequency table.

C++

```
candidate = none
count = 0

for each x in nums:
    if count == 0:
        candidate = x

    if x == candidate:
        count++
    else:
        count--

return candidate
```

STATE

candidate = current survivor

BALANCE

count = unmatched votes

OUTPUT

final candidate

4. 🔍 Walkthrough — first half

Array: [2, 2, 1, 1, 1, 2, 2]

i=0

x=2

cand=2
count=1

i=1

x=2

cand=2
count=2

i=2

x=1

cand=2
count=1

i=3

x=1

cand=2
count=0

Two 1s cancel two 2-votes, but the majority has more to spare.

Walkthrough — second half

When count reaches zero, the next value can become the candidate.

i=4

x=1

cand=1
count=1

i=5

x=2

cand=1
count=0

i=6

x=2

cand=2
count=1

Final candidate = 2 

Edge cases

ONE ELEMENT

$[7] \rightarrow 7$

ALL SAME

$[3,3,3] \rightarrow 3$

SMALL MAJORITY

$[2,2,1] \rightarrow 2$

NO GUARANTEE

If existence is not guaranteed, verify the candidate.

5. C++ Implementation

Interview-ready LeetCode 169 solution.

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int candidate = 0;
        int count = 0;

        for (int x : nums) {
            if (count == 0) candidate = x;

            if (x == candidate)
                count++;
            else
                count--;
        }

        return candidate;
    }
};
```

Code breakdown

What each variable is really doing.

candidate

The current possible majority.

count

The current unmatched vote balance.

count == 0

There is no active leader → choose x.

x != candidate

One opposing vote cancels one candidate vote.

Important: count is NOT the total frequency.



6. 🕒 Time & Space Complexity

TIME

$O(n)$

Each array element is processed once.

SPACE

$O(1)$

Only candidate and count are stored.

WHY IT MATTERS

This is the key advantage over a hash-map solution.

Why does it work?

A short proof intuition.

FACT 1

Every decrement pairs one candidate vote with a different value.

FACT 2

Removing different pairs does not change who has the most votes.

CONCLUSION

Because the majority starts with more than half the votes, it cannot be eliminated by pair cancellation. The final survivor is the majority.

7. LeetCode Practice

Click the links to practice.

169 • Majority Element

Core problem — Boyer–Moore

[Solve ↗](#)

229 • Majority Element II

Find elements appearing more than $n/3$

[Solve ↗](#)

275 • H-Index II

Bonus pattern practice — binary search

[Solve ↗](#)



Mini challenge

Try this before looking at the answer.

INPUT

[3, 3, 4, 2, 3, 3, 5]

QUESTION

What are candidate + count at the end?

HINT

Track only two things: current candidate and its balance. Do not count frequencies.

Answer appears on the next slide →

Mini challenge — answer

[3, 3, 4, 2, 3, 3, 5]

FINAL CANDIDATE

3

FINAL COUNT

1

CHECK

3 appears 4 times out of 7 $\rightarrow 4 > 3.5$, so 3 is the majority.



8. 💡 Interview Tip

Explain the reasoning, not just the code.

BEST 20-SECOND EXPLANATION

“I keep a candidate and a balance. Equal values increase the balance; different values decrease it. Different pairs cancel, and because the majority has more than half the votes, it survives.”

COMPLEXITY

$O(n)$ time • $O(1)$ space

FOLLOW-UP

Is a majority guaranteed?

9. Revision + Tomorrow

Finish Day 17 with the pattern locked in.

REMEMBER

Majority $> n/2$

Same $\rightarrow +1$

Different $\rightarrow -1$

Zero $\rightarrow$ new candidate

COMPLEXITY

Time $O(n)$

Space $O(1)$

Core idea:
cancellation

TOMORROW

DAY 18

Next DSA pattern

Keep the streak going 

GitHub [Add your GitHub ↗](#)

Replace the placeholder link with your actual GitHub profile before posting.

